

PRODUCT OVERVIEW

MemGuard

AI Agent 的記憶可觀測性、安全與稽核證據層

看見 Agent 記住了什麼、哪些記憶曾在輸出生成時可用、狀態如何演變，以及記憶是否出現衝突或可疑修改。

專案快照：MemguardV1

整理日期：2026 年 8 月 1 日

一頁摘要

一句話定位 MemGuard 是 AI Agent 的「記憶黑盒飛行記錄器」：它不替換原有記憶系統，而是攔截每一次記憶操作，再組成可查詢、可比較、可稽核的證據鏈。

你正在建立的不是另一個向量資料庫，也不是普通 Prompt Log，而是一層專門針對 Agent Memory 的 observability 與 governance infrastructure。它讓團隊回答四個問題：

- Agent 在某一刻讀了、寫了、更新了或刪除了哪些記憶？
- 某個輸出生成時，有哪些持久化記憶證據可被使用？之後又寫出了什麼新狀態？
- 同一記憶是否被不同 Agent 在短時間內互相覆寫，或出現疑似污染 / 指令注入？
- 能否把技術事件轉成開發者可除錯、合規人員可審查的時間線與報告？

現在已經具備的產品形態

目前程式庫已形成一個可展示、可本機部署的端到端產品雛形：Python SDK 負責攔截；FastAPI Control Plane 負責接收、儲存與分析；Next.js Dashboard 負責呈現；Docker Compose 串起 PostgreSQL、Keycloak、後端與前端；FinCompli 提供金融合規示範案例。

產品的分層願景

層級	主要使用者	要回答的問題	現況
Tier 1 除錯	AI 工程師	是哪一段記憶造成這次行為？	核心能力已落地
Tier 2 可觀測性	平台團隊	記憶如何變化、是否穩定？	事件、時間線、Diff、統計已具雛形
Tier 3 可稽核性	風控 / 合規	能否用證據解釋一次輸出？	Decision Trace 與模板式報告已落地
Tier 4 治理	CISO / 管理層	如何把記憶當作組織風險面？	中長期藍圖

為什麼這個產品值得做

Agent 的記憶已經成為新的隱藏狀態

傳統應用的錯誤多半可以從輸入、程式碼與資料庫重現；但具有長期記憶的 Agent 會被跨回合、跨工具、跨 Agent 的狀態影響。當答案錯了，僅看 Prompt 與 LLM 回應，往往看不到真正參與流程的歷史記憶、SOP、使用者偏好或工作狀態。

三個高價值痛點

- 除錯困難：開發者知道輸出錯了，卻不知道 Agent 當時讀到哪個舊狀態、是否讀到衝突版本。
- 安全風險：惡意內容一旦被寫入長期記憶，可能在未來多次被重新讀取，形成延遲生效的 memory poisoning。
- 合規缺口：高風險決策需要可追溯證據，但普通日誌沒有以記憶實體、版本、讀寫關係與租戶界線組織。

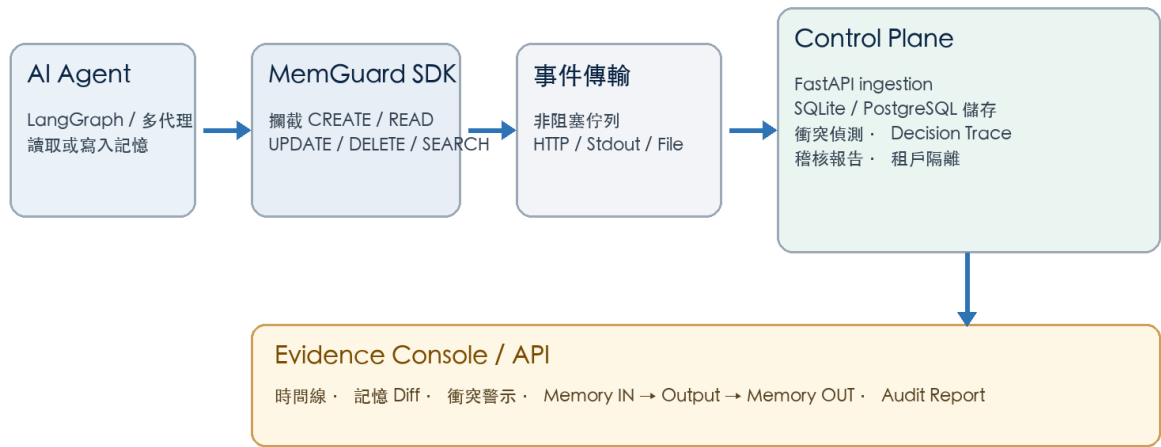
產品洞察 在微服務世界，Distributed Tracing 讓請求路徑可見；MemGuard 的主張是：在 Agent 世界，Memory Tracing 會成為同樣基礎的能力。

它刻意不做什麼

- 不取代 Mem0、Redis、向量資料庫或 LangGraph Checkpointer 等既有記憶後端。
- 不把所有 LLM 請求代理到自己的服務；目前重點是記憶操作與證據關聯。
- 不宣稱「記錄到的記憶」等同模型內部真正的因果解釋；它提供的是可驗證的 evidence lineage。

系統如何運作

MemGuard 的資料流：從記憶操作到可稽核證據



設計原則：MemGuard 觀察並記錄記憶狀態，不取代原有記憶後端，也不代理 LLM 呼叫。

圖 1 | MemGuard 端到端資料流

核心流程

1. Agent 透過既有記憶機制進行 CREATE、READ、UPDATE、DELETE、QUERY 或 SEARCH。
2. MemGuard SDK / 框架 Adapter 在操作邊界產生 MemoryEvent，記錄 Agent、Session、Namespace、記憶鍵、類型、時間、Hash 與上下文。
3. Transport 以背景佇列非同步送出事件；觀測後端不可用時，設計目標是不阻塞 Agent 主流程。
4. Control Plane 驗證並持久化事件，同時建立 Decision Trace、衝突分析、統計與稽核資料。
5. Evidence Console 把事件轉為時間線、狀態差異、輸出前證據、輸出後寫入與報告。

兩個最重要的資料單位

資料單位	代表什麼	核心欄位 / 關聯
MemoryEvent	一次原子記憶操作	event_id、agent/session/namespace、operation、memory_key/type、before/after、content_hash、timestamp、context
DecisionTrace	把一次 Agent 輸出前後的證據串起來	input_event_ids → prompt/output hash 與摘要 → output_event_ids ；另可附 evidence ranking / influence score

目前已實作的產品能力

1. SDK 與整合

- 可安裝的 Python SDK，核心包含 MemoryEvent、Interceptor、DecisionTrace 與 Influence 計算。
- LangGraph Checkpointer Adapter 可包裝既有 checkpointer，攔截 get / put 類型操作。
- Stdout、HTTP、File 與 Null Transport；HTTP 支援背景佇列、批次、重試與 flush。
- 事件傳送採 best-effort / fire-and-forget 思路，觀測失敗不應中斷業務 Agent。

2. Control Plane 與資料

- FastAPI 提供事件 ingestion、記憶查詢、時間線、Decision Trace、衝突分析、Session、統計與稽核 API。
- 本機預設可使用 SQLite；Pilot / Compose 路徑支援 PostgreSQL 16。
- Keycloak OIDC Bearer Token、tenant claim 與 API tenant enforcement 已進入主架構。

3. 分析與呈現

- Memory Timeline 與 operation / agent 篩選。
- Memory Diff：顯示 before / after 以及欄位變動。
- Conflict Detection：辨識不同 Agent 在短時間內修改同一記憶鍵。
- Decision Trace：呈現 Memory IN → Agent Output → Memory OUT，並保留缺失證據警示。
- Audit Report：以模板產生 compliance、debug 或 business 風格摘要。

隱私與證據 SDK 的資料模型以 content hash 為核心追蹤值；原文內容可以按整合與部署策略保留或最小化。Hash 能證明內容版本是否改變，但不是資料防洩漏的完整替代品。

示範場景：FinCompli 金融合規 Agent

專案內的 FinCompli baseline 是 MemGuard 的垂直示範：多個 Agent 共同分析一宗疑似拆分交易（structuring）案例。MemGuard 的價值不是替合規 Agent 下判斷，而是把判斷過程中的記憶證據留下來。

示範故事

- 1. Fraud Detection Agent 讀取客戶 / 交易歷史，產生 fraud analysis。
- 2. Case History Agent 搜尋過去 SAR 案例，取得相似案例與相似度。
- 3. Compliance Research Agent 查閱規例 / SOP 記憶。
- 4. Report Generation Agent 綜合工作記憶，產出 FILE SAR 或其他處置建議。
- 5. MemGuard 把讀取、搜尋、工作記憶寫入與最終輸出串成可審查鏈路。

同一份證據，服務三種角色

角色	他看到的價值	典型問題
AI 工程師	事件時間線、狀態 Diff、缺失鏈路	哪個 checkpoint 或記憶版本讓結果偏掉？
平台 / 資安	衝突、租戶隔離、異常寫入與操作量	是否有跨 Agent 覆寫、污染或資料邊界問題？
合規 / 業務	輸入證據、輸出摘要、後續寫入與 Audit Report	這個建議有什麼持久化證據可供人員覆核？

對外講法 Without MemGuard：『AI 標記了這筆交易，但我們很難重建它當時看到的記憶。』
With MemGuard：『我們可以逐項查看輸出生成時已持久化的案例、規例與工作狀態，以及輸出後寫入了什麼。』

目前成熟度、限制與下一步

可合理宣稱的現況

面向	現在可以說	暫時不要過度承諾
產品	可本機展示的端到端 Memory Observability MVP	已是完整企業治理平台
整合	LangGraph Adapter 與通用 Interceptor / Transport 基礎	已支援所有主流 Agent / Memory Framework
證據	可重建持久化事件與輸出前後 lineage	能證明模型內部真正的因果推理
安全	OIDC、租戶隔離、衝突與簡單可疑內容規則	已具完整 RBAC、策略引擎、不可竄改帳本與 SIEM
稽核	可產生模板式 session report	已完成法規級自動合規判定
部署	Compose : Keycloak + PostgreSQL + FastAPI + Next.js	已完成多區域、高可用、雲端托管與 SLA

最重要的工程限制

- 目前主 SDK Adapter 以 LangGraph 為核心；Mem0、AutoGen、CrewAI、Zep 等仍屬擴展方向。
- Influence score / evidence ranking 是解釋性排序訊號，不應包裝成已被科學驗證的模型因果貢獻值。
- 衝突偵測目前主要依記憶鍵、Agent 與時間窗規則；語義衝突、版本合併與自動修復仍可深化。
- 稽核摘要以模板生成為主；LLM 增強敘事在程式中保留接口，但不是完整交付能力。
- 這次環境缺少 pytest 套件，因此本文以程式碼、Docker 配置與既有測試檔判斷功能，未重新跑完整測試套件。

建議的下一階段

1. 產品聚焦：把『Output → Evidence → Memory Writes』調查工作流做成唯一主故事。
2. 證據可信度：加入不可竄改事件鍵、簽章、保留策略與 evidence completeness 指標。
3. 框架覆蓋：優先補 Mem0，再依客戶選擇 AutoGen / CrewAI / Zep。
4. 治理能力：增加 policy engine、RBAC、告警、人工覆核與 SIEM / ticketing 整合。
5. 企業部署：完成 migration、備份還原、壓力測試、自動化 CI 與雲端拓撲。

技術版圖與快速理解

主要技術組件

區塊	技術 / 目錄	責任
SDK	sdk/memguard	事件模型、攔截、LangGraph Adapter、Transport、Influence
Backend	backend/app · FastAPI	ingestion、查詢、分析、稽核、OIDC tenant enforcement
Database	SQLite / PostgreSQL	MemoryEvent、DecisionTrace 與 migration
Identity	Keycloak	OIDC 登入、JWT、tenant claim
Frontend	frontend · Next.js	Evidence Console、Timeline、Diff、Conflict、Audit
Demo	demo_simple.py / demo_with_dashboard.py / fincompli-baseline	終端、Dashboard 與金融合規案例
Deployment	docker-compose.yml	本機 / Pilot 四服務編排

如果要向別人介紹這個專案

30 秒版本 MemGuard 是 AI Agent 的記憶可觀測性與安全層。它記錄 Agent 對長期與工作記憶的每一次讀寫，並把這些事件串成輸出前後的證據鏈。這讓工程師能除錯、資安團隊能發現衝突或污染、合規人員能覆核高風險 Agent 輸出。

投資 / 產品版本 當 Agent 開始依賴跨回合記憶，記憶就變成新的企業風險面。MemGuard 想成為這一層的 Datadog + audit trail：先從除錯與 evidence console 切入，再走向政策、權限、保留與治理。

一句話結論

你正在把『AI 為什麼這樣做』從模糊的模型說法，轉成可查驗的記憶事件、版本差異與證據鏈。這個切入點既有立即的工程價值，也能延伸到安全與合規治理。

依據範圍

本文件依據 2026-07-31 工作區快照整理，主要參考 README、SDK / Backend / Frontend 程式碼、Docker Compose、技術設計、產品規格、測試檔與 Dashboard redesign spec。較早期進度文件互相存在狀態差異，因此本文優先採用目前程式碼中可直接辨識的能力。